SW Engineering CSC648-848 Spring 2026

GatorBazaar

Team 07

Members	
Team Lead	Dylan Cordoni (dcordoni@sfsu.edu)
Github Maintainer	Abhay Bajaj
Back-End Lead	Sebastian Cervantez
	Ryan Zhang
Front-End Lead	Daniel De Jesus
	Kenny Camarra
	Diego Rivera-Soto

Milestone 4

History Table	
Date Submitted	May 20, 2026

1. Product Summary

Product Name: GatorBazaar

Describe your product: GatorBazaar provides a simple web-based platform where users can list items they no longer need and browse items posted by others. Listings are organized into categories such as textbooks, electronics, clothing, and dorm essentials, and many other features we have coming down the pipeline, making it easy for users to quickly find what they are looking for. Students can post items for sale, upload descriptions and prices, and connect with buyers within the SFSU community. Professors can also use the platform to share or sell academic materials or recommend resources to students. By centralizing these exchanges in one place, GatorBazaar helps reduce waste, saves students money, and creates a convenient and safe way to reuse items within the campus community.

Itemized list of ALL major committed functions (your final priority 1 or P1 list):

Priority 1:

- Unregistered User
  - The system shall allow visitors to access the marketplace homepage.
  - The system shall allow visitors to browse publicly available item listings.
  - The system shall allow visitors to search listings using keywords.
  - The system shall allow visitors to view item details including title, description, price, and images.
  - The system shall allow visitors to filter listings by category.
- Registered User
  - The system shall allow users to register using an SFSU email address.
  - The system shall allow users to receive notifications when they receive messages.
  - The system shall verify that registered users are associated with an SFSU email domain.
- Seller
  - The system shall allow registered users to create listings for items they wish to sell.
  - The system shall allow users to upload images when creating listings.
  - The system shall allow users to mark listings as sold.
  - The system shall allow users to delete their listings.
- Buyer

        - The system shall allow users to initiate the sending of messages to other users
        - regarding listings.
    - Admin
        - The system shall allow administrators to remove listings that violate policies.
        - The system shall allow administrators to monitor marketplace activity.

URL to your product accessible to instructors: http://3.150.110.210

2. Usability test plan for selected function

1. Test objectives: - The purpose of this usability is to evaluate how users can search for and browse marketplace listings within the web application.. The test focuses on the search bar, category filtering, and viewing item listings. The objective is to determine if the users can easily find items of interest, understand the UI, and navigate with ease.

2. Test background and setup

System Setup:

The system being tested is from our web application marketplace. Developed for students to buy, sell, and browse different kinds of items such as textbooks, furniture, electronics, school supplies, and apartment rooms search. The tester will access the application through a web browser of their choice.

Staring Point:

The tester will begin in our homepage when they visit our website. The tester should not be required to have an account to browse through the marketplace, however would be required to Log in if the tester wants to purchase an item.

Hardware Requirements:

The tester should have a stable internet connection, a laptop or desktop computer, keyboard and mouse, and a web browser installed.

Intended Users:

The intended users are SFSU students who want to browse, search, buy, or sell second-hand items within their campus community. Users who have varying levels of technical experience may use our web application.

URL of the System:

http://3.150.110.210

Test Environment:

The usability test will be conducted remotely from the tester's home environment. No cameras or screen recording are required. The tester may be monitored. The tester will not receive training before beginning the test. We will collect data after the tester completes their test.

3. Usability Task description:

The tester should complete the following tasks in order and must read all instructions carefully before beginning.

Task 1 - Search for an item

1. Search an item
2. Review the search results.
3. Find the item you would consider purchasing

Task 2 - Browse by Category

1. Change the category of your preferred item
2. Go through different categories
3. Click on the item to review

Task 3 - Review Listing Information

1. Find the item price
2. Determine the item condition or description
3. Message the seller

4. Plan for evaluation of Effectiveness:

Effectiveness will be measured by observing whether users can successfully complete the assigned tasks without assistance. The evaluation will focus on whether users can locate items, browse categories, and access listing information correctly. User errors and failed task attempts will also be recorded during testing. A task will be considered effective if the users can complete it successfully without error.

5. Plan for Evaluation of efficiency:

Efficiency will be evaluated by measuring how quickly users are able to complete the assigned tasks while using the web application marketplace. The evaluation will consider the amount of time required to locate items, browse listings, and access listing information. The number of unnecessary actions, repeated attempts, and navigation mistakes made by tester during testing will also be recorded. A task will be considered efficient if the user can complete it smoothly with minimal effort.

6. Plan for Evaluation of user satisfaction (Likert scale questionnaire):

Statement	Strongly Disagree	Disagree	Neutral	Agree	Strongly Agree

The search function was easy to use	1	2	3	4	5
The category navigation was clear and understandable	1	2	3	4	5
I was able to find item information quickly and easily	1	2	3	4	5
I would feel comfortable using this marketplace application regularly	1	2	3	4	5

7. GenAI use: Did not use

Note: we strongly recommend that team members NOT involved in UX use the above subjective Likert scale usability evaluation and provide answers and feedback in the template defined in this subsection. Use this feedback to improve UX of your application. This is optional but strongly recommended.

3. QA test plan and QA testing

1. Test objectives: - The objective of this QA testing is to verify that the marketplace search and browsing functionality operates correctly across different web browsers. The testing focuses on ensuring that users can successfully search for listings, browse categories, and view listing details without functional errors or broken navigation.

2. HW and SW setup (including URL):

HW Setup:

1. Laptop or desktop computer
2. Keyboard and mouse
3. Stable internet connection

SW Setup

1. Operating Systems: Windows/MacOs/Linux
2. Browser 1: Google Chrome
3. Browser 2: Microsoft Edge
4. Local development server running Vite

URL:

http://3.150.110.210

3. Feature to be tested

1. Search bar functionality
2. Category Filtering
3. Item listing display
4. Navigation to item details page

4. QA Test plan: in table format:

Test	**Title**	**Description**	**Input**	**Expected correct output**	**Chrome result**	**Edge result**
1	Search for item	Verify that users can search for listings using keywords	Enter “textbook” into search bar	Relevant textbook listings are displayed	PASS	PASS
2	Category filtering	Verify that category buttons correctly filter listings	Select “furniture” category	Only furniture relevant items are displayed	PASS	PASS
3	View item listing	Verify that users can open and view listing details	Click on a listing card	Item detail page loads with title, price, description, picture,	PASS	PASS

5. You also must perform the testing as per the plan above on 2 major WWW

browsers of your choice and record the results in a form above (PASS or FAIL).
The QA test plan was executed using both Google Chrome and Microsoft Edge browsers. All major marketplace search and browsing functions worked with ease and without errors in both browsers. No crashes, glitches, broken links, display issues occurred. Search functionality, category filtering, and item listing navigation behaved as expected across both testing environments.

6. GenAI use: Did not use AI

4. Peer Code Review:

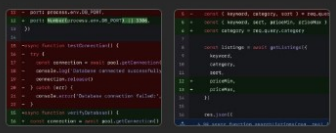
Sebastian Emilliano Cervantez

Reply | Reply all | Forward

To: Ryan Zhang; Abhay Bajaj; Kenny Camarra; Dylan Jon Alec Cordoni; Daniel De Jesus; Diego Fernando Rivera-Soto

Wed 5/20/2026 6:41 PM

You replied on Wed 5/20/2026 6:51 PM



2 attachments (886 KB) Save all to OneDrive - San Francisco State University Save All Attachments

Requesting code review implementation on backend files below.

- Fast fail start implementation
- searchController.js
- SearchModel.js

Sebastian Emiliano Cervantez

...

RZ

Ryan Zhang

Reply | Reply all | Forward

To: Sebastian Emilliano Cervantez

Wed 5/20/2026 6:51 PM

Hey Sebastian,

Overall, good work!

General themes of what I saw when I went over your code:

You had header comments at the top of the code, which is great. But they unfortunately didn't have any inline commenting, which would be really helpful when others are trying to read your code. I suggest that you include short comments about a function when you complete it, like what inputs, what it does, what it returns.

You do a good job of validating and sanitizing inputs, which is critical in catching errors. There was one issue where in the searchModel file, you used an embedded userId instead of the placeholder ?. This use can be prone to SQL injections, so I recommend that you use the '?' instead. Otherwise, you practiced good security practices.

RZ

Ryan Zhang

Reply | Reply all | Forward

To: Sebastian Emilliano Cervantez

Wed 5/20/2026 6:51 PM

Hey Sebastian,

Overall, good work!

General themes of what I saw when I went over your code:

You had header comments at the top of the code, which is great. But they unfortunately didn't have any inline commenting, which would be really helpful when others are trying to read your code. I suggest that you include short comments about a function when you complete it, like what inputs, what it does, what it returns.

You do a good job of validating and sanitizing inputs, which is critical in catching errors. There was one issue where in the searchModel file, you used an embedded userId instead of the placeholder ?. This use can be prone to SQL injections, so I recommend that you use the '?' instead. Otherwise, you practiced good security practices.

You also generally following our naming conventions, which is great. There was one instance where you used item instead of listing in the controller file, but otherwise everything is consistent.

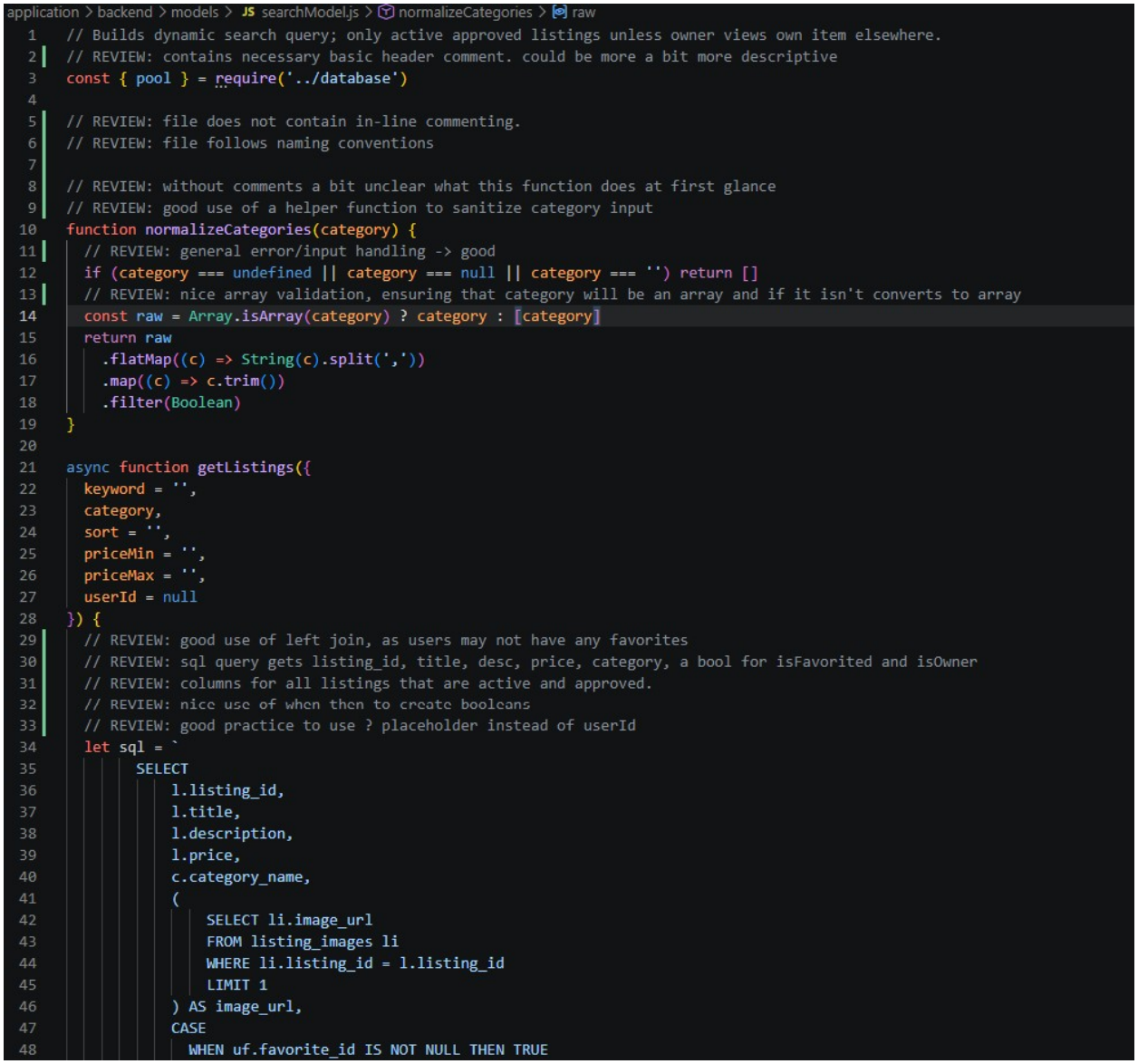
There was also a bug where is session doesn't exist, the req.session.userId check will crash, so instead you should probably use session?.userId.

Otherwise, I think you did great work here. Just some fixes to the SQL input and adding more comments and I think we'll be great.

Thanks,
Ryan

...

Reply | Forward


application > backend > models > searchModel.js > normalizeCategories > raw
// Builds dynamic search query; only active approved listings unless owner views own item elsewhere.
// REVIEW: contains necessary basic header comment. could be more a bit more descriptive
const { pool } = require('../database')
// REVIEW: file does not contain in-line commenting.
// REVIEW: file follows naming conventions
// REVIEW: without comments a bit unclear what this function does at first glance
// REVIEW: good use of a helper function to sanitize category input
function normalizeCategories(category) {
// REVIEW: general error/input handling -> good
if (category === undefined || category === null || category === '') return []
// REVIEW: nice array validation, ensuring that category will be an array and if it isn't converts to array
const raw = Array.isArray(category) ? category : [category]
return raw
.flatMap((c) => String(c).split(','))
.map((c) => c.trim())
.filter(Boolean)
}
async function getListings({
keyword = '',
category,
sort = '',
priceMin = '',
priceMax = '',
userId = null
}) {
// REVIEW: good use of left join, as users may not have any favorites
// REVIEW: sql query gets listing_id, title, desc, price, category, a bool for isFavorited and isOwner
// REVIEW: columns for all listings that are active and approved.
// REVIEW: nice use of when then to create booleans
// REVIEW: good practice to use ? placeholder instead of userId
let sql = `
SELECT
l.listing_id,
l.title,
l.description,
l.price,
c.category_name,
(
SELECT li.image_url
FROM listing_images li
WHERE li.listing_id = l.listing_id
LIMIT 1
) AS image_url,
CASE
WHEN uf.favorite_id IS NOT NULL THEN TRUE

```
            WHEN uf.favorite_id IS NOT NULL THEN TRUE
            ELSE FALSE
          END AS isFavorited,
          CASE
            WHEN l.user_id = ${userId} THEN TRUE
            ELSE FALSE
          END AS isOwner
        FROM listings l
        JOIN Categories c ON l.category = c.category_id
        LEFT JOIN user_favorites uf
          ON uf.listing_id = l.listing_id
          AND uf.user_id = ${userId}
        WHERE l.active = 1 AND l.approval_status = 'Approved'`

  const params = []
  // REVIEW: good use of SQL placeholders to prevent SQL injection here
  const kw = typeof keyword === 'string' ? keyword.trim() : ''
  if (kw) {
    sql += ' AND (l.title LIKE ? OR l.description LIKE ?)'
    params.push(`%${kw}%`, `%${kw}%`)
  }

  const categories = normalizeCategories(category)
  // REVIEW: code doesn't add filtering unless categories exists or is not empty good
  if (categories.length > 0) {
    const placeholders = categories.map(() => '?').join(', ')
    sql += ` AND LOWER(TRIM(c.category_name)) IN (${placeholders})`
    params.push(...categories.map((c) => c.toLowerCase()))
  }


  if (priceMin !== '' && priceMin !== undefined && priceMin !== null) {
    const n = Number(priceMin)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price >= ?'
      params.push(n)
    }
  }
  if (priceMax !== '' && priceMax !== undefined && priceMax !== null) {
    const n = Number(priceMax)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price <= ?'
      params.push(n)
    }
  }
```

```
  const categories = normalizeCategories(category)
  // REVIEW: code doesn't add filtering unless categories exists or is not empty good
  if (categories.length > 0) {
    const placeholders = categories.map(() => '?').join(', ')
    sql += ` AND LOWER(TRIM(c.category_name)) IN (${placeholders})`
    params.push(...categories.map((c) => c.toLowerCase()))
  }


  if (priceMin !== '' && priceMin !== undefined && priceMin !== null) {
    const n = Number(priceMin)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price >= ?'
      params.push(n)
    }
  }
  if (priceMax !== '' && priceMax !== undefined && priceMax !== null) {
    const n = Number(priceMax)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price <= ?'
      params.push(n)
    }
  }

  if (sort === 'price_low') {
    sql += ' ORDER BY l.price ASC'
  } else if (sort === 'price_high') {
    sql += ' ORDER BY l.price DESC'
  } else if (sort === 'newest') {
    sql += ' ORDER BY l.created_at DESC'
  } else {
    sql += ' ORDER BY l.created_at DESC'
  }

  const [rows] = await pool.query(sql, params)
  return rows
}
// REVIEW: code regularly checks for valid input and sanitizes inputs, good practice
module.exports = {
  getListings,
}
```

```
// Public search: keyword, category, price range, sort. Favorites flag if session exists.
// REVIEW: contains necessary basic header comment
const { getListings } = require('../models/searchModel')
// REVIEW: file missing inline commenting

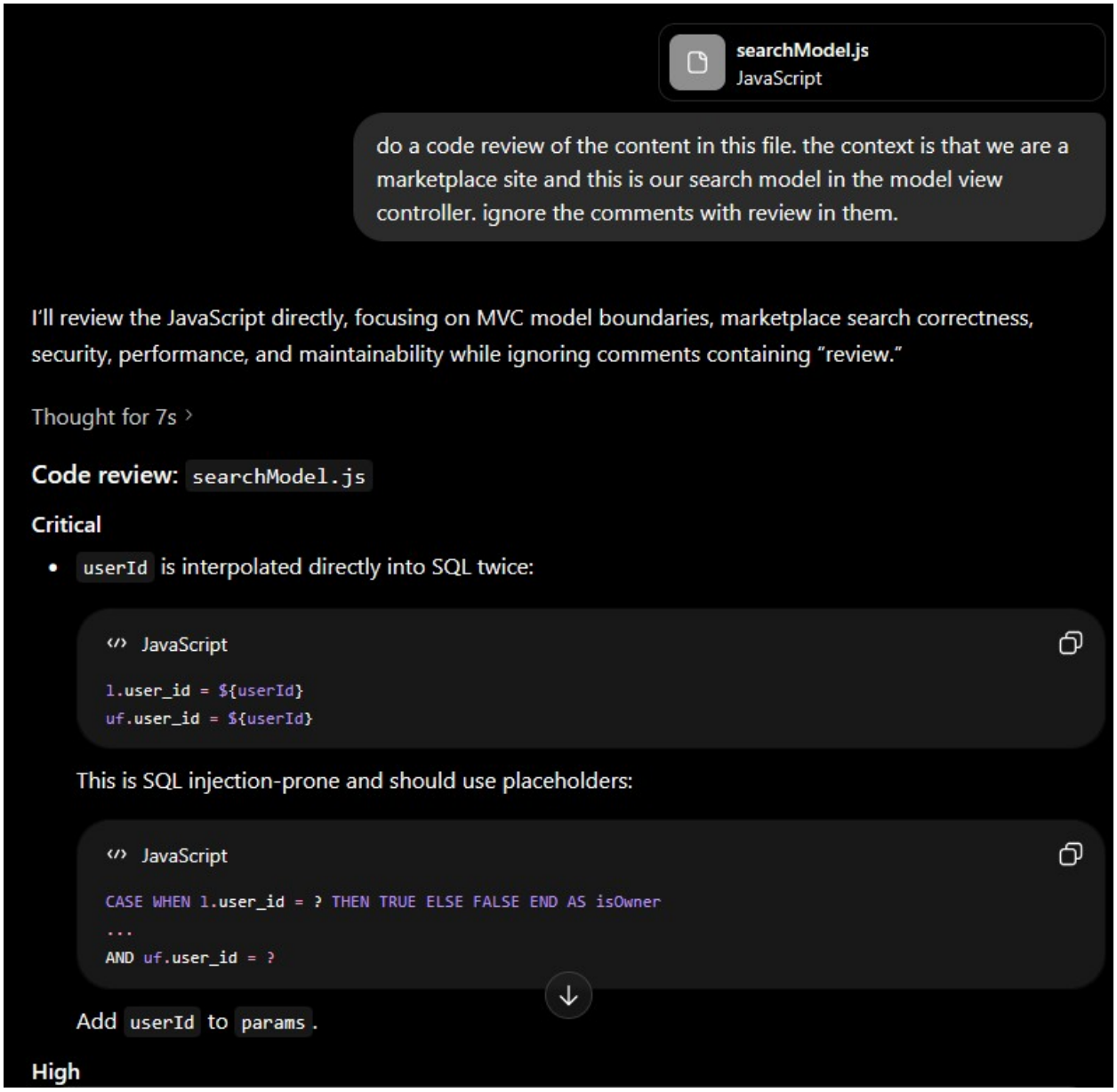
// REVIEW: generally follows naming conventions
// REVIEW: handles errors using try catch model
async function searchListings(req, res) {
  try {
    const { keyword, sort, priceMin, priceMax } = req.query
    const category = req.query.category
    // REVIEW: req.session.userId can crash if session is undefined
    const userId = req.session.userId || null

    const listings = await getListings({
      keyword,
      category,
      sort,
      priceMin,
      priceMax,
      userId
    })
    // REVIEW: should use listings here instead of items
    res.json({
      items: listings,
      total: listings.length,
    })
  } catch (err) {
    console.error(err)
    res.status(500).json({ error: err.message })
  }
}

module.exports = {
  searchListings,
}

```

GenAI code review session



- No pagination or result limit. Marketplace search can become slow and expensive as listings grow. Add `LIMIT` / `OFFSET`.
- `LIKE '%keyword%'` will not scale well. Consider full-text search for title/description.
- The image subquery has no ordering:

```SQL
LIMIT 1
```

  This returns an arbitrary image. Use something deterministic, such as `ORDER BY li.is_primary DESC, li.created_at ASC`.

Medium

- `LOWER(TRIM(c.category_name))` may prevent index usage. Prefer normalizing category names in the database or filtering by `category_id`.
- Price filters silently ignore invalid values. That may hide bad client input. Consider validating earlier and returning a clear error.
- No handling for `priceMin > priceMax`.
- Negative prices are accepted unless the database prevents them.
- `isFavorited` may duplicate listing rows if `user_favorites` allows multiple rows for the same user/listing. Add a unique constraint or use `EXISTS`.

Low

- Table name casing is inconsistent:

```SQL
JOIN Categories c
```

  If the rest of the schema uses lowercase names, this can break on case-sensitive systems.
- `TRUE` / `FALSE` may come back as `1` / `0` depending on the MySQL driver. Normalize in JS if the API

expects booleans.

- `sort = ''` defaults to newest, which is fine, but this could be simplified.

Suggested fix for the biggest issue

JavaScript

```
const params = [userId, userId]

let sql = `
  SELECT
    l.listing_id,
    l.title,
    l.description,
    l.price,
    c.category_name,
    (
      SELECT li.image_url
      FROM listing_images li
      WHERE li.listing_id = l.listing_id
      ORDER BY li.is_primary DESC, li.created_at ASC
      LIMIT 1
    ) AS image_url,
    CASE WHEN uf.favorite_id IS NOT NULL THEN TRUE ELSE FALSE END AS isFavorited,
    CASE WHEN l.user_id = ? THEN TRUE ELSE FALSE END AS isOwner
  FROM listings l
  JOIN Categories c ON l.category = c.category_id
  LEFT JOIN user_favorites uf
    ON uf.listing_id = l.listing_id
    AND uf.user_id = ?
  WHERE l.active = 1 AND l.approval_status = 'Approved'
`
```

Overall: the structure is reasonable for an MVC model, but the direct `userId` interpolation is the main blocker.

searchController.js
JavaScript

do a code review of this file as well, also ignoring REVIEW comments

Thought for a few seconds ›

Code review: `searchController.js`

High

- `req.session.userId` can throw if `req.session` is undefined:

```JavaScript
const userId = req.session?.userId || null
```

- The API exposes internal error messages:

```JavaScript
res.status(500).json({ error: err.message })
```

  Use a generic client response and log the real error server-side:

```JavaScript
res.status(500).json({ error: 'Search failed' })
```

Medium

- Query params are passed through without validation or normalization. Validate `priceMin`, `priceMax`, `sort`, and `category` before calling the model.
- `total: listings.length` only reports the numb[illegible]returned, not the total matching count. This becomes misleading once pagination is added.
- No pagination params are accepted. Marketplace search should support `limit` and `offset` or `page`.

Low

- `category` is destructured separately from the other query params for no clear reason:

```
const { keyword, sort, priceMin, priceMax, category } = req.query
```

- `console.error(err)` is acceptable for simple apps, but a structured logger is better in production.

Suggested revision

```
async function searchListings(req, res) {
  try {
    const {
      keyword,
      category,
      sort,
      priceMin,
      priceMax,
      limit = 20,
      offset = 0,
    } = req.query

    const userId = req.session?.userId || null

    const listings = await getListings({
      keyword: keyword?.trim(),
      category,
      sort,
      priceMin,
      priceMax,
      limit: Number(limit),
      offset: Number(offset),
      userId,
    })

    res.json({
      items: listings,
      total: listings.length,
    })
  } catch (err) {
    console.error('Search listings failed:', err)
    res.status(500).json({ error: 'Search failed' })
  }
}
```

Main issue: this controller trusts query params too much and leaks server errors to the client.

We used ChatGPT Edu, the ChatGPT provided by SFSU.

We prompted it to do a code review on the contents of searchModel.js and searchController.js, the backend implementation of our search function. We also used it to review certain SQL syntax. It was helpful as a look into what modern safe coding practices are, since as students, we may not be the most familiar with what safety practices real developers use in the software, as we're all still learning. This also gave us quick insights into what could possibly be vulnerabilities or bugs in our programs and how to address them if they exist.

For example, we prompted it with "do a code review of the content in this file. the context is that we are a marketplace site and this is our search model in the model view controller. ignore the comments with review in them. " and attached the file. ChatGPT told us that the use of ${userId} was dangerous because it didn't protect against SQL injection, which was an area of safety concern we went over in class.

Another example is after the GenAI code review, we prompted it with "explain the danger of sql injection and the use of embedded userId more", asking it to go into further detail. It explained that even though it looks harmless, if someone could influence userId, they could change the SQL query. It gave an example of if userId was 5 OR 1=1, it could return all the users in the table.

Utility of GenAI: Medium

5. Self-check on best practices for security – ½ page

Asset to be Protected	Types of Possible/Expected Attacks	Consequence of Security Breach	Strategy to Mitigate/Protect the Asset
User Password	Credential theft, DB leak, brute force	Account takeover, users reuse password on other sites leading to more harm	Passwords must never be stored in plain text. Use Salted hashing before saving to the DB. Enforce strong password rules.
User Registration Data (email, profile info)	Data exposure, code injection from fields, account enumeration	privacy breach, unauthorized access, harm to reputation	Validate and sanitize inputs, enforce SFSU domain, avoid error messages that give away unnecessary information.

Database	SQL injection, unauthorized DB access, stolen credentials	Data corruption & deletion, data leak, service outages	Use parameterized queries, principle of least-privilege, no DB credentials in source code, restrict DB network access.
Endpoints	Insecure endpoints, bad JSON payloads, CORS misuse	Unauthorized cross-site calls, crashing, unexpected behavior in page serving	Validate request schemes, handle errors without giving information to the end-user, keep dependencies updated, keep CORS restricted to a known origin.
Credentials	Leakage of sensitive/useful data/log-in information from Git history	Compromised Database access and all contents.	Can store credentials in a .env locally and make sure to never commit real credentials, can rotate/change credentials if needed, use different credentials for the dev & production versions.

Confirmation of PW encryption in the DB:

We confirm that we encrypted the passwords in the database with bcrypt.

Confirmation of Input data validation (what is being validated and what code you used):

What We Validate:	**Code Used To Validate:**
Email used in Registration	const emailValid = form.email.endsWith('@sfsu.edu')
Full Name	const fullNameValid = form.fullName.trim().length > 0
Username	const usernameValid = /^[A-Za-z]{5,}$/.test(form.username
Password	const passwordValid = form.password.length >= 8 && /[0-9]/.test(form.password) && /[!@#$%^&*]/.test(form.password)
Password Confirmation Match	const passwordsMatch = form.confirmPassword.length > 0 && form.password === form.confirmPassword
SearchInput is 40 Characters	<input className="gb-search-input" type="text" placeholder={placeholder} value={query} maxLength={40} onChange={(e) => onQueryChange(e.target.value)}

	aria-label={searchInputAriaLabel} autoFocus={autoFocus} />
Terms&Conditions are accepted	<input type="checkbox" name="agreedToTerms" checked={form.agreedToTerms} onChange={handleChange} />

6. Self-check of the adherence to original Non-functional specs

1. DONE - Application shall be developed, tested and deployed using tools and cloud servers approved by Class CTO and as agreed in M0
2. DONE - Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. ISSUE All our selected application functions shall be rendered well on mobile devices (no native app to be developed)

   Unfortunately, we did not have the time to make the site render on well on mobile device as our time was spent making the site functional and implementing P1 functionality.
4. DONE Posting sales information and messaging to sellers shall be limited only to SFSU students
5. DONE Critical data shall be stored in the database on the team's deployment server.
6. DONE No more than 50 concurrent users shall be accessing the application at any time
7. DONE Privacy of users shall be protected
8. DONE The language used shall be English (no localization needed)
9. DONE Application shall be very easy to use and intuitive
10. DONE Application shall follow established architecture patterns
11. DONE Application code and its repository shall be easy to inspect and maintain
12. DONE Google analytics shall be used
13. DONE No e-mail clients or chat services shall be allowed. Interested users can only message to sellers via in-site messaging. One round of messaging (from user to seller) is enough for this application
14. DONE Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. DONE Site security: basic best practices shall be applied (as covered in the class) for main data items
16. DONE Media formats shall be standard as used in the market today
17. DONE Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools

18. DONE The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Spring 2026. For Demonstration Only” at the top of the WWW page Nav bar. (Important to not confuse this with a real application). You must use this exact text without any editing.